ResQRoute

Intelligent Surplus & Emergency Aid Coordination Platform

PRODUCT REQUIREMENTS DOCUMENT

Version 1.0 | Hackathon MVP | October 2026

Product vision

Make every safe, usable donation count by getting the right aid to the right destination, using the right transport, before its deadline.

Core proposition

A real-time coordination platform connecting surplus donors, volunteer riders, and receiving shelters. It treats expiry, transport capacity, recipient demand, volunteer reliability, resource compatibility, and changing field conditions as live operational constraints.

Core capability	Purpose
Time-aware dispatch	Prioritize feasible deliveries before applicable safety deadlines.
Capacity-aware allocation	Split oversized donations across eligible riders and shelters.
Safety enforcement	Block unsafe, incompatible, or over-capacity deliveries.
Automatic recovery	Release stale reservations and recover from destination or rider changes.
Judge simulation lab	Let evaluators trigger failures and inspect actual state changes.

Prepared as a build-oriented PRD emphasizing a working prototype, testable business rules, and reproducible judge demonstrations.

Document contents

- 1. Executive summary and problem definition
- 2. Goals, scope, users, and prioritization
- 3. Functional requirements FR-01 through FR-10
- 4. Intelligent dispatch algorithm and feasibility rules
- 5. UI/UX requirements and screen specifications
- 6. Technical architecture, events, data model, and APIs
- 7. Shipment state machine and data integrity invariants
- 8. Judge simulation lab and acceptance scenarios
- 9. KPIs, non-functional requirements, implementation roadmap
- 10. GitHub workflow, testing strategy, demonstration plan, risks, and definition of done

1. Executive summary

ResQRoute is a real-time coordination platform that connects surplus donors, volunteer riders, and receiving shelters to redistribute perishable food and essential aid safely, efficiently, and transparently.

Unlike a conventional donation listing application, ResQRoute treats expiry, transport capacity, recipient demand, volunteer reliability, resource compatibility, and changing field conditions as live operational constraints.

The platform must answer five questions: what aid needs to move now; which riders can transport it within the safety window; how to split batches across riders and shelters; what to do when a volunteer disappears or demand changes; and whether a safe recovery is possible.

Product hypothesis: a real-time operational state machine combined with deadline-aware allocation, capacity-constrained dispatch, reservation expiry, and event-triggered recovery can reduce avoidable failures versus manual coordination. This is a hypothesis to validate in simulation. The problem statement's reported waste percentage should not be presented as independently verified research without a source.

Core value proposition

- Time-aware dispatch based on remaining safe handling time and estimated delivery duration.
- Capacity-aware allocation across multiple riders and eligible shelters.
- Safety enforcement for deadlines, compatibility, storage, and recipient dependencies.
- Automatic recovery after stale reservations, rider failures, and demand changes.
- Transparent decisions with reasons, audit events, and reproducible simulations.

2. Problem definition

Operational failure	Consequence	Required response
Donation exceeds rider capacity	Untransported surplus	Split into capacity-compliant shipments.
Shelter demand is smaller than supply	Oversupply or spoilage	Allocate across eligible shelters.
Volunteer claims but disappears	Donation remains locked	Expire stale claims and reopen allocation.
Food expires before arrival	Unsafe delivery	Reject infeasible routes.
Shelter demand changes in transit	Stranded delivery	Revalidate and safely reroute.
Recipient lacks required resources	Aid cannot be used effectively	Enforce dependency rules.
Several volunteers claim the same donation	Duplicate dispatch	Use atomic reservation and assignment rules.

Goals

- Unify donation inventory, rider availability, and shelter demand.
- Prevent double-booking, ghost reservations, and over-capacity assignments.
- Deliver only when routes satisfy applicable safety limits.
- Reallocate unassigned aid when demand changes.
- Recover in-transit shipments through safe, auditable rerouting.
- Demonstrate critical scenarios through working controls and visible state changes.

Non-goals for the first prototype

- Nationwide production logistics, real payment processing, or fully autonomous emergency dispatch.
- Medical diagnosis or clinical decision-making.
- Guaranteed road ETAs without a routing provider.
- Unrestricted public registration and identity verification.

3. Target users and roles

Donor

- Create batches with item type, quantity, location, expiry/preparation details, pickup deadline, handling requirements, and restrictions.
- View allocation, pickup, and delivery status.
- Withdraw a batch not yet collected, subject to safety and dispatch rules.

Volunteer rider

- Set availability, carrying capacity, location, and supported item types.
- Accept or decline eligible assignments; confirm arrival, pickup, departure, and delivery.
- Report delays, vehicle issues, or inability to continue.
- Receive replacement assignments or return instructions when needed.

Shelter or receiving center

- Publish demand by item category and quantity.
- Set intake capacity, operating status, and resource readiness (water, fuel, refrigeration, storage).
- Confirm incoming allocations and received quantities.
- Update demand or close intake while shipments are in transit.

Operations administrator

- Monitor live network and exception queue.
- Review allocation recommendations and resolve failures.
- Pause unsafe dispatches and inspect audit logs and performance.
- Run controlled failure simulations for judges.

For the hackathon, roles can be represented by a role switcher, but the backend must enforce role-specific permissions.

4. Scope and priorities

Priority	Scope	Deliverables
P0 — Must work	Core logistics engine	Donation management, demand tracking, capacity splitting, compatibility, atomic claims, reservation timeout, state machine, recovery.
P1 — Visibility	Operational UX	Map/schematic, shipment timelines, urgency indicators, exception alerts, audit trail, impact dashboard.
P2 — Advanced	Enhancements	External routing, forecasting, multilingual UI, notifications, historical analytics.

5. Functional requirements

FR-01 — Donation batch management

- Fields: unique batch ID; item category; quantity and unit; pickup location; created time; pickup deadline; safe-use deadline; handling requirements; dependencies; status.
- Acceptance: valid batch gets unique ID and appears immediately; invalid safety fields block dispatch; quantity cannot be negative or over-allocated; expired batches cannot be assigned; safety-relevant edits trigger revalidation.
- Distinguish donor-declared expiry from a validated safety deadline. Do not assume food is safe until a displayed timestamp.

FR-02 — Shelter demand and compatibility

- Maintain item-specific demand, remaining quantities, intake capacity, open/closed status, resource availability, restrictions, and last-updated timestamp.
- Acceptance: allocations never exceed remaining demand or capacity; closed shelters receive no new assignment; missing dependencies block dispatch; demand changes trigger revalidation; concurrent assignments cannot overfill demand.

FR-03 — Rider management and transport capacity

- Track rider ID, availability, current/last-known location, capacity, handling capabilities, active assignments, heartbeat, and reliability state.
- Acceptance: no over-capacity assignment; unavailable riders receive no new jobs; heartbeat timeout flags inactivity; a returning rider reconciles active shipment state before new work.

FR-04 — Intelligent allocation and shipment splitting

- Consider safety deadline, pickup/delivery time, quantity, shelter demand/capacity, rider capacity, route feasibility, compatibility, reliability, and fallback options.
- Example: 180 meals, rider capacity 40, Shelter A demand 50. Allocate no more than 50 to A and no more than 40 to each rider. Allocate the remainder elsewhere only when feasible.
- Acceptance: shipment quantities reconcile to allocated quantity; every shipment respects rider capacity; shelter demand is not exceeded; infeasible plans are rejected; the UI explains selection and rejection reasons.

FR-05 — Ghost reservation prevention

- Reservations are temporary locks with server-side expiry. Lifecycle: AVAILABLE → RESERVED → PICKED_UP → IN_TRANSIT → DELIVERED, with eligible RELEASED, EXPIRED, or CANCELLED outcomes.
- Require acknowledgement and pickup confirmation within configured windows. Release unpicked quantity atomically on expiry; retry with eligible riders; reject late responses to old reservations; record repeated expiry.
- Reservation creation, expiry, and release must be transactional or otherwise concurrency-safe. Browser refresh timers alone are insufficient.

FR-06 — Live tracking and timeline

- Show shipment status, last update, rider, origin, destination, quantity, ETA, and remaining safe handling time when known.
- Timeline events: assignment offered/accepted, arrival, pickup, departure, delay, demand change, reroute proposal/approval, delivery, failure, or safe return.
- Simulation may use a simulated clock and seeded locations, but must be labelled and must update actual application state.

FR-07 — In-transit demand changes and recovery

- On destination closure or demand becoming zero: detect change; recompute feasible alternatives; exclude closed/incompatible shelters; check deadline; verify capacity and demand; propose route; authorize; notify rider; audit old/new destination.
- If no feasible alternative exists, enter RECOVERY_REQUIRED. Never pretend a collected shipment is still at the donor; preserve physical location and custody.

FR-08 — Safety and dependency enforcement

- Central validator must be used during assignment, acceptance, pickup, and rerouting.
- Check item eligibility, recipient demand/capacity, storage/water/fuel requirements, rider capability, deadline feasibility, and current shipment state.
- If the safe-use deadline cannot be met, block dispatch. Regulated medical handling rules must come from a qualified authority, not be invented by the algorithm.

FR-09 — Notifications and exception queue

- Show actionable alerts: reservation expiry, missing heartbeat, at-risk pickup, infeasible deadline, filled demand, no fallback, closed destination, unmet dependency, or capacity conflict.
- Each exception includes severity, entity, timestamp, reason, and safe available actions.

FR-10 — Audit trail

- Record actor/system component, entity and shipment IDs, previous/new state, timestamp, reason, validation outcome, and reroute/reassignment reference.
- The audit trail must reconstruct why an item was assigned, released, blocked, or rerouted.

6. Intelligent dispatch algorithm

Pipeline

- Observe network state: batches, riders, demand, deadlines, and resources.
- Validate hard constraints: safety, capacity, compatibility, availability, demand.
- Generate feasible plans: split batches and match riders to destinations.
- Rank feasible plans by urgency, travel time, demand fit, reliability, and fallback feasibility.
- Reserve atomically, track progress, and recompute after relevant changes.

Hard constraints versus preferences

Hard constraints are non-negotiable. Reject routes that exceed capacity, target closed shelters, violate dependencies, or cannot meet the safe deadline. Ranking preferences choose among feasible options only; a high score can never compensate for a safety violation.

Deadline feasibility

Let Ts be remaining time until the applicable safe-use deadline; Tp be time to pickup; Td be delivery time after pickup; Th be handling/handoff time; and Tb be the safety buffer.

$$Tp + Td + Th + Tb < Ts$$

Use a consistent time source. If estimates are missing or too uncertain, disclose that limitation and do not represent the route as guaranteed safe.

Transparent ranking model

Normalize factors to 0–1: urgency U, demand match D, rider reliability R, fallback feasibility F, and delivery efficiency E.

$$S = 0.35U + 0.25D + 0.15R + 0.15F + 0.10E$$

This is a configurable starting point, not a validated optimum. Document normalization, tune with simulation, and expose score breakdowns.

Recommended MVP algorithm

- Sort unallocated batches by urgency.
- Generate compatible shelters with remaining demand.
- Find available riders with sufficient capacity.
- Estimate pickup and delivery feasibility.
- Rank valid rider-destination combinations.
- Allocate the largest feasible quantity without exceeding constraints.
- Repeat until allocated or no feasible combination remains; record why residual quantity remains unallocated.

Recomputation triggers

- New batch; shelter demand/status change; rider unavailable; reservation expiry; route infeasibility; shipment delay; destination closure.

Recompute only affected allocations where possible, preserving valid active shipments and avoiding unnecessary churn.

7. UI/UX requirements

Dashboard

- Top KPIs: active batches, shipments in transit, available riders, open exceptions.
- Live map or schematic showing donors, riders, shelters, and shipment routes.
- Priority exception queue with clear severity and direct actions.
- Recent activity and a detail panel for the selected shipment.

Required screens

Screen	Content	Primary actions
Operations dashboard	KPIs, map, alerts, live activity	Inspect exception, open shipment
Donation management	Batches, deadlines, allocation	Create, edit, withdraw
Create donation	Validated safety/dependency form	Save batch
Shelter management	Demand, capacity, resources, status	Update demand, close intake
Rider management	Availability, capacity, location, jobs	Activate, deactivate, flag issue
Dispatch center	Allocation plans, score, feasibility	Assign, approve, reject
Shipment details	Rider, destination, deadline, timeline	Pickup, report delay, reroute
Exception center	Blocked, delayed, stale, failed jobs	Resolve, retry, escalate
Simulation lab	Scenario controls, clock, seeded entities	Run, pause, reset
Audit and impact	History, deliveries, failure reasons	Filter, export

Design system

- Professional civic-tech aesthetic, light theme, low clutter.
- Deep teal for primary actions; slate for neutral data; blue for in-transit; amber for at-risk; red for blocked/critical; green for verified completion.
- Readable sans-serif, clear hierarchy, concise tables and cards, responsive sidebar, details in panels/pages.
- Every action shows success, validation error, or explicit rejection reason.
- Keyboard navigation, visible focus, sufficient contrast, and text labels in addition to color.
- Use strong colors sparingly so true failures stand out. Never show success before the underlying mutation succeeds.

8. Technical architecture

Recommended stack

Layer	Recommendation	Responsibility
Frontend	React, Vite, JavaScript, CSS, Lucide React	Role-based views, dashboard, map, simulation controls
Backend	Node.js, Express, Socket.IO	Auth, validation, dispatch, reservations, recovery
Database	PostgreSQL, Prisma ORM	Batches, demand, reservations, shipments, events, audit
Jobs	Server scheduler or durable job mechanism	Reservation expiry and deadline checks
Simulation	Injectable deterministic clock and seeded data	Repeatable judge scenarios
Routing	Adapter with seeded estimates initially	Pickup/delivery feasibility; optional external provider

Architecture rules

- Backend is source of truth; validate all critical transitions server-side.
- Use transactions, conditional updates, and idempotency keys for reservations and quantity changes.
- Enforce timeouts on the server; emit Socket.IO events only after committed state changes.
- Persist audit events with critical operations; inject simulation clock into business logic.
- For a short sprint, SQLite is acceptable initially if allocation and validation remain separate from UI.

Suggested real-time events

Event	Trigger / response
batch.created	New batch; refresh eligible inventory
allocation.proposed	Plan found; display recommendation
reservation.created	Lock committed; update available quantity
reservation.expired	Release eligible quantity and retry
rider.unresponsive	Flag affected jobs
demand.updated	Revalidate allocations
shipment.reroute_proposed	Show safe alternatives
shipment.rerouted	Update destination and notify rider
shipment.delivered	Update demand and impact totals
shipment.blocked	Show reason and record event

These event names are proposed contracts. Document payload schemas, authorization, and error responses before integration.

9. Data model and integrity

Entity	Important fields / relationships
User	id, name, role, status; donor, rider, shelter, or operator
DonationBatch	donorId, itemType, quantity, deadlines, status; multiple shipments
Shelter	location, openStatus, capacity; demand records
ShelterDemand	shelterId, itemType, requestedQty, fulfilledQty, version
ShelterResource	shelterId, resourceType, available, verifiedAt
RiderProfile	userId, maxCapacity, availability, heartbeatAt
Reservation	batchId, riderId, quantity, expiresAt, status
Shipment	batchId, riderId, shelterId, quantity, status
ShipmentEvent	shipmentId, eventType, timestamp, payload
AuditLog	actorId, action, entityId, previousState, newState
SimulationRun	scenario, clockMode, status, seed

Required invariants

Quantity conservation: every batch quantity must belong to exactly one accounting category—unallocated, reserved, active shipment, completed, or another explicitly accounted state. Cancelled/returned quantities must be reconciled.

$Q_{batch} = Q_{unallocated} + Q_{reserved} + Q_{active} + Q_{completed} + Q_{other\ accounted\ states}$

$\text{Sum(active shipment quantity for rider)} \leq \text{rider capacity}$

$\text{Accepted and committed quantity} \leq \text{shelter remaining demand}$

Idempotency: repeating pickup, delivery, or expiry requests must not change quantity twice. Concurrency: simultaneous claims for the last quantity cannot both succeed.

10. API requirements

Method / endpoint	Purpose
POST /api/batches	Create donation
GET /api/batches	List/filter donations
PATCH /api/batches/:id	Update eligible fields
POST /api/shelters	Register receiving center
PATCH /api/shelters/:id/demand	Update demand and capacity
PATCH /api/shelters/:id/status	Open/close intake
POST /api/riders/:id/heartbeat	Update rider activity
PATCH /api/riders/:id/availability	Change rider status
POST /api/dispatch/plan	Generate feasible allocation proposals
POST /api/reservations	Atomically reserve quantity
POST /api/reservations/:id/accept	Accept assignment
POST /api/reservations/:id/release	Release eligible reservation
POST /api/shipments/:id/pickup	Confirm pickup
POST /api/shipments/:id/deliver	Confirm delivery
POST /api/shipments/:id/report-issue	Report delay/failure
POST /api/shipments/:id/reroute	Validate/apply reroute
GET /api/shipments/:id/events	Fetch timeline
GET /api/operations/summary	Dashboard metrics
POST /api/simulation/scenarios/:id/run	Run seeded scenario
POST /api/simulation/reset	Restore demo baseline

Every mutation must validate authorization and state, return structured errors, reject invalid transitions, protect against duplicate requests, persist relevant audit events, and notify clients only after commit.

Suggested error codes: INSUFFICIENT_QUANTITY, RIDER_CAPACITY_EXCEEDED, SHELTER_CLOSED, DEPENDENCY_NOT_MET, DEADLINE_INFEASIBLE, RESERVATION_EXPIRED, NO_SAFE_DESTINATION.

11. Shipment state machine

Normal path: AVAILABLE → RESERVED → ACCEPTED → PICKED_UP → IN_TRANSIT → DELIVERED.

Exception states include EXPIRED/RELEASED for unpicked reservations, REROUTE_PENDING for route validation/approval, RECOVERY_REQUIRED when no approved feasible route exists, and CANCELLED/FAILED with explicit reconciliation.

- Delivered shipments cannot be reopened by stale requests.
- Expired reservations cannot be accepted.
- Picked-up shipments cannot be released back to donor inventory as though physically present there.
- Rerouting preserves custody and quantity records.
- Delivery confirmation updates shelter fulfillment exactly once.
- Every exceptional transition has a reason.

12. Judge simulation lab

The simulation lab is a first-class feature. Judges should trigger failures, see actual application state change, inspect outcomes, and reset to a reproducible baseline.

A — Oversized donation and multiple riders

Donation 180 meals; rider capacity 40; shelter demands 50, 70, and 60. Expected: shipments respect rider capacity and shelter demand; all 180 are allocated only if enough riders/routes are feasible; otherwise residual quantity is reported.

B — Ghost reservation

Rider A reserves but does not respond; Rider B is available. Expected: visible countdown, server-side expiry, atomic release, reassignment if still feasible, rejection of Rider A's late response, and safe stop if deadline is no longer feasible.

C — Shelter closes in transit

Rider carries 40 meals for Shelter A; A closes; B is open and compatible. Expected: A removed from candidates, custody preserved, B validated, reroute proposed/approved, timeline updated.

D — Dependency mismatch

Ration kits require water and fuel; A lacks fuel, B has resources and demand. Expected: A blocked with DEPENDENCY_NOT_MET; B eligible if all other checks pass; no match means unassigned plus exception.

E — Safety deadline breach

15 minutes remain, but every route exceeds remaining safe time including buffer. Expected: all routes blocked, no normal assignment, clear safety exception, no normal override.

F — Simultaneous claims

Two riders claim last 20 meals concurrently. Expected: one reservation succeeds; the other receives insufficient quantity; balance never goes negative.

G — Rider failure after pickup

Rider reports vehicle issue while carrying a shipment. Expected: exception workflow, custody preserved, replacement carrier/destination validated before handoff.

H — No fallback destination

Original shelter closes; alternatives are full, closed, or incompatible. Expected: RECOVERY_REQUIRED, rejection reasons displayed, intervention requested, no fabricated delivery.

Simulation controls

- Scenario selector, run/pause/reset, accelerated/normal/manual clock, configurable playback speed.
- Seeded entities and repeatable data; event feed and state inspector.
- The actual backend services and business rules must be reused; do not build a separate fake simulation path.
- Clearly label simulated locations, ETAs, time, and outcomes.

13. KPIs and evaluation

KPI	Definition	MVP target
Safe allocation rate	Feasible allocated quantity / eligible quantity offered	Measure baseline; maximize without constraint violations
On-time delivery	Deliveries completed before applicable deadline / deliveries due	Track by scenario
Capacity violations	Accepted assignments over rider capacity	0
Demand/capacity violations	Accepted allocations exceeding shelter constraints	0
Unsafe dispatches	Infeasible/unsafe deliveries authorized	0
Ghost reservation recovery	Expired unpicked reservations released/reconciled correctly	100% in tests
Duplicate allocation	Quantity allocated more than once	0
Stale mutation	Invalid/stale requests that mutate state	0
Reroute success	Feasible reroute scenarios recovered correctly	100% in deterministic tests
No-fallback detection	Infeasible recovery cases flagged correctly	100% in deterministic tests
Audit completeness	Critical transitions with audit event	100%

Impact dashboard

- Meals safely delivered; aid kits delivered; on-time deliveries.
- Quantity awaiting allocation; quantity blocked for safety/compatibility.
- Reservations released and reassigned; shipments recovered after destination changes.
- Confirmed waste avoided only when supported by evidence.

Do not count a created donation, proposed allocation, or pickup as a successfully delivered meal. Targets are prototype correctness goals, not claimed real-world results.

14. Non-functional requirements

Area	Requirement
Reliability	Critical operations preserve valid state under failures and retries
Security	Role-based authorization; no unrestricted privileged mutations
Performance	Target p95 under 500 ms for ordinary seeded-data operations, excluding external routing
Real-time	Target visible event propagation under 1 second on healthy local demo
Scalability	Avoid full-network recomputation for unrelated events
Consistency	Transactional quantity, capacity, and reservation enforcement
Observability	Structured logs, event IDs, decision reasons, error reporting
Accessibility	Keyboard-operable workflows and readable status indicators
Responsiveness	Desktop-first with functional tablet/mobile layouts
Privacy	Minimize personal data and unnecessary continuous location collection
Recoverability	Seed, reset, and replay scenarios reproducibly
Maintainability	Centralized business rules and shared API contracts

Targets should be measured in the prototype and adjusted to the environment; they are not production guarantees.

15. Implementation roadmap

Phase	Work	Exit criterion
1. Foundation	Workspace, schema, seed data, dashboard shell, API contracts, reset	App launches, loads seeded entities, displays real states
2. Validation	Donation/shelter/rider CRUD, safety/dependency checks, state machine, audit	Invalid assignments rejected by backend tests
3. Allocation	Dispatch algorithm, splitting, atomic reservations, expiry, heartbeat	Oversized donation and ghost reservation tests pass
4. Recovery	Socket.IO, timelines, exception queue, demand revalidation, reroute approval	Shelter closes in transit; UI reflects recovery
5. Judge readiness	Eight scenarios, clock controls, reset, responsive UI, metrics, concurrency tests	Judge can trigger scenarios without developer intervention

16. GitHub repository and continuous commits

Suggested repository structure

```
resqroute/ ■■■ frontend/src/{components,pages,features,services,styles}/ ■■■
backend/src/{modules,events,middleware}/ ■■■ shared/contracts/ ■■■ prisma/schema.prisma ■■■
tests/{unit,integration,scenarios}/ ■■■ docs/{PRD.md,API.md,DEMO.md} ■■■ .env.example ■■■ .gitignore ■■■
README.md
```

Commit message	Meaning
chore: initialize frontend and backend workspace	Runnable setup
feat: add donation, shelter and rider models	Seed data and migrations
feat: implement safety and capacity validation	Validation tests
feat: implement deadline-aware dispatch	Splitting and explanations
feat: add atomic reservations and expiry	Ghost reservation tests
feat: implement shipment tracking and rerouting	State transitions and audit
feat: add interactive failure simulation lab	Scenarios and reset
test: cover concurrency and safety edge cases	Regression coverage
docs: add setup guide and judge demonstration	Reproducible setup

Commit small, real, reviewable changes and push regularly. Keep the default branch runnable, use feature branches for parallel work, and never commit secrets, API keys, or real beneficiary data.

17. Testing strategy

Unit tests

- Deadline feasibility; capacity and quantity; dependency matching; score/tie-break rules; expiry; state transitions; reroute eligibility.

Integration tests

- Two riders claim same quantity concurrently.
- Reservation expiry and reassignment occur once.
- Delivery updates demand once.
- Shelter closes while shipment is in transit.
- Rider becomes unavailable after pickup.

- Expired batch cannot be dispatched.
- Reroute preserves custody and quantity.
- Failed transaction leaves no partial allocation.

End-to-end tests

- Create donation; see plan; accept assignment; advance clock; trigger failure; inspect exception/recovery; verify final database-backed state and audit trail.

A green dashboard is not proof of correctness. Tests must assert resulting state and quantity balances.

18. Judge demonstration plan (5–7 minutes)

Time	Demo step	What to prove
0:00–1:00	Introduce dashboard and problem	Real operational coordination, not a static donation listing
1:00–2:30	Create/load 180-meal donation	Batch splitting, rider capacity, shelter demand
2:30–3:30	Trigger ghost reservation	Expiry, reassignment, stale-action rejection
3:30–5:00	Close shelter during transit	Revalidation, reroute proposal, updated timeline
5:00–7:00	Trigger safety/dependency failure	Blocked operation, reason, audit and impact metrics

Test the demo from a clean reset. Avoid depending on external network, live maps, or real volunteer accounts for the core demonstration.

19. Risks and mitigations

Risk	Mitigation
Scope too large	Finish state machine and five critical scenarios before advanced features
Fake frontend success	Backend responses authoritative; show errors explicitly
Race conditions	Transactions, conditional updates, idempotency
Simulation differs from production	Reuse same business rules and backend services
Unsafe rerouting/handoff	Preserve custody; validate destination before reassignment
Uncertain food deadlines	Separate declared/validated deadlines; block infeasible routes
Map provider fails	Clearly labeled seeded routing mode
Opaque AI recommendations	Transparent deterministic scoring and hard constraints
Misleading impact metrics	Count confirmed shipment events; label simulation data
Inconsistent demo state	Deterministic reset and repeatable seeds

20. Definition of done

Core functionality

- Create/manage donations, riders, shelters.
- Split donations without exceeding capacity.
- Enforce demand, capacity, and dependencies.
- Prevent duplicate/expired reservations.
- Recover from unresponsive riders and reject stale actions.

Dynamic recovery

- Close shelter during transit.
- Find/validate feasible alternative.
- Show explicit exception if none exists.
- Preserve custody, quantity, audit history.

Safety and integrity

- Block shipments that miss safety deadline.
- Pass concurrency and idempotency tests.
- Verify quantity/demand invariants after each scenario.

Judging readiness

- Run scenarios through UI.
- Reset and replay reliably.
- Show accurate impact metrics and audit trail.
- Publish setup, API contracts, tests, and regular commits.

21. Final product recommendation

ResQRoute should stand out through operational intelligence and recovery—not by implementing every possible logistics feature.

- Deadline-aware allocation: only propose routes that satisfy relevant safety constraints.
- Ghost-reservation recovery: release stalled claims and reassign eligible quantities safely.
- Dynamic destination recovery: react to demand changes after pickup.
- Compatibility enforcement: prevent aid from going to sites that cannot use it.
- Judge-controlled failure simulation: let evaluators trigger real state changes and verify results.
- Transparent decision-making: explain every allocation, rejection, and recovery action.

> **A polished map is useful, but a correct recovery engine is more important. A sophisticated score is useful, but hard safety constraints matter more. The strongest prototype can fail safely, recover intelligently, and prove what happened.**

Immediate next step: confirm the repository and existing stack, implement the data model and state machine, then complete the three highest-priority judge scenarios end to end before expanding the UI.